AI Compiler Performance Engineering

Auto-Loops Books — ch25

Contents

1 Auto-Optimization and AI-Assisted Compilation **1**

1.1 RL autotune 2

1.2 HW reward 6

1.3 YiRage auto 10

1.4 Key Takeaways 13

1.5 Conclusion 14

Bibliography **15**

Chapter 1

Auto-Optimization and AI-Assisted Compilation

When a decode compiler exposes thousands of tile sizes, fusion boundaries, and backend flags, hand-tuning one schedule per SKU stops scaling—yet blind autotune that optimizes GEMM FLOPs alone still ships schedules that lose on bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). Part VI established that each backend (XLA, TVM, Triton, MLIR) owns a different search grammar; this chapter asks how *auto-optimization* should search when the objective is decode goodput across GPU, CPU, and NPU fleets, not a single microbench winner (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing host overhead—yet cannot fix illegal HBM traffic (Chen, 2026; NVIDIA, 2024). An autotuner that rewards median kernel time without residency legality will pick launches that spill KV every token—exactly the failure mode Chapter 1 framed as "fast kernels, slow decode" (Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023).

Three mechanisms stack: **RL-guided search** over compiler knobs when the space is too large for exhaustive grid search; **hardware-aware reward functions** that score bytes/token, launches/token, and legality on the target tier; and **YiRage-style integrated search** that binds IR metadata, backend registry, and fingerprint caches so auto-optimization does not fork a second toolchain (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). The sections below treat each layer as a production contract—what to measure, what to forbid in the reward,

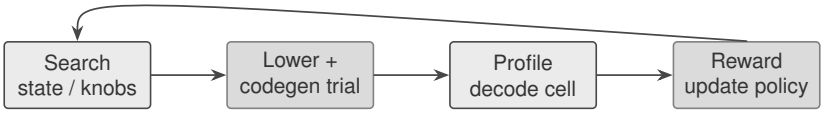


Figure 1.1: RL autotune loop: compiler knobs → trial schedule → decode-profiled reward → policy update (not peak GEMM alone).

and when AI-assisted compilation helps versus hides regressions (SemiAnalysis, 2024; AMD, 2024a; Goodfellow *et al.*, 2016).

This chapter sits at the end of Part VII for a reason: you cannot auto-tune what you cannot measure, and you cannot trust measurement that ignores residency (Chen, 2026; Dao *et al.*, 2022; Davies *et al.*, 2025). Earlier chapters gave you backends and benchmarks; here the compiler team operationalizes *search* under decode SLOs (Chen, 2020; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Treat autotune artifacts—policies, fingerprints, profile scripts—with the same rigor as source code: versioned, reviewed, rollback-ready (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026, 2020).

Anti-pattern catalog: (i) optimizing prefill-only benchmarks for decode fleets; (ii) single-number rewards that hide bytes/token debt; (iii) external autotuners that bypass compiler legality checks; (iv) promoting GPU winners to NPU fleets without re-profile (Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The rest of the chapter turns each anti-pattern into a concrete gate (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016).

1.1 RL autotune

RL autotune enters when the Cartesian product of tile sizes, pipeline stages, and fusion predicates exceeds what grid search or genetic algorithms can explore before your release train departs (Chen, 2020; Goodfellow *et al.*, 2016). Classical TVM/Ansor-style autotuning treats the schedule as a structured program with measurable features; RL layers add a policy that proposes knob vectors conditioned on IR shape, target SKU, and prior trial outcomes (Chen, 2020; Chen and YiRage Contributors, 2026). The production mistake is training that policy on prefill

shapes or batch-32 training graphs while deployment runs BS=1 decode—the search converges to schedules that never touch launches/token (Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023).

Search spaces diverge by hardware class even when the math is identical. GPU trials must respect shared-memory caps, TMA legality, and CUDA Graph capture constraints (NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024). CPU trials swap tile axes for cache lines and NUMA pinning; reward spikes from L3 residency can invert when KV grows past LLC (Goodfellow *et al.*, 2016; Chen, 2020). NPU/XDNA trials are compile-time static: illegal dynamic buffers fail before profiling, so RL exploration needs a *legality oracle* ahead of expensive simulation (AMD, 2024b,a). One global search space imported from GPU codegen is how teams burn weeks on trials that never legalize on the fleet they ship (AMD, 2024b; Chen and YiRage Contributors, 2026).

Ansor and auto-scheduler literature established *program sampling*—mutate loop orders, split factors, and cache levels under measured cost models (Chen, 2020; Goodfellow *et al.*, 2016). RL replaces random mutation with a learned proposal distribution when cost models lie on decode shapes (thin MLPs, small batch GEMMs, attention with paged KV) (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen, 2026; Davies *et al.*, 2025). The handoff from classical autotune to RL is not "smarter magic"—it is better exploration when analytic models miss orchestration (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Keep classical grid search as sanity check on small knob subsets (Chen, 2020, 2026; Vellaisamy *et al.*, 2026).

Decode-aware RL should treat each trial as a *cell*: fixed context bucket, BS=1, steady-state after warmup, metrics logged as ms/token *and* bytes/token *and* launches/token (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Prefill-only rewards overweight arithmetic intensity; decode rewards expose orchestration cliffs when graph replay is absent (NVIDIA, 2024; Chen, 2026). MoE and speculative paths add non-stationary branches—policies must either exclude those subgraphs from the MDP or include explicit state for routing entropy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Operational guardrails: cap trial wall-clock per layer block; freeze RNG seeds for reproducible CI; store *fingerprinted* schedules keyed by IR hash + SKU + context bucket (Chen and YiRage Contributors, 2026; Chen, 2020). When RL proposes a schedule that improves GEMM but regresses bytes/token, the reward function failed—not the hardware (Chen, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024). YiRage encodes iron rules from Chapter 2 as checkable IR metadata so

Knob family	GPU decode focus	CPU / NPU delta
Tile / block	Shared mem, TMA stages	Cache line, static tile slots
Fusion depth	Graph capture legality	DMA chain length
Backend choice	Triton vs CUDA vs CUTLASS	oneDNN / AIE kernel gen
Objective	min launches/token	min bytes + DMA edges

Table 1.1: RL autotune search axes differ by target; shared math does not imply shared knob grammar.

illegal trials fail fast before GPU hours burn (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Multi-hardware delta. Hopper RL may explore TMA double-buffer stages; XDNA policies must emit static tile assignments with bounded DMA depth; CPU policies optimize cache blocking with OpenMP gang sizes—identical transformer math, incomparable action spaces (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

Example 1.1.1. Decode cell (BS=1, ctx=2048): an RL policy that reduces MLP tile width may cut shared memory but force an extra KV spill—reward must penalize bytes/token, not only kernel duration (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026).

Industrial teams keep a *human prior* schedule as RL baseline: never accept a policy update that fails A/B against the prior on all three counters (Davies *et al.*, 2025; Chen, 2020, 2026). AI-assisted compilation is not autonomy—it is search acceleration with decode metrics as the constitution (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016; AMD, 2024a).

Feature engineering for RL policies should mirror what Ansor-style cost models already encode—loop extent, reuse distance, parallel axis—but add decode-specific state: context length bucket, graph-capture eligibility, and whether KV is paged or dense (Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024). Without those features the policy collapses to "pick bigger tiles" because that wins microbench GEMMs on prefill-shaped tensors (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). Export the feature vector alongside every trial so postmortems can explain *why* a schedule won, not only that loss decreased (Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Exploration versus exploitation trade-offs show up in fleet rollout: ϵ-greedy trial injection can destabilize p99 when a bad schedule briefly serves traffic (Davies

et al., 2025; Kwon *et al.*, 2023). Canary pools per SKU—10% of decode replicas run candidate schedules with automatic rollback on SLO breach—keep RL useful without gambling the whole cluster (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026). Offline RL from logged profiles is safer when experimentation budget is thin; online RL requires hard legality masks (Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016).

Part VI compiler chapters (XLA, TVM, Triton) each expose different knob surfaces; RL autotune should treat backend choice as part of the action space, not a pre-search constant (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). A policy that only tunes Triton tile sizes while XLA would legalize a cheaper fusion is incomplete search (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Document backend-specific action spaces in runbooks so ML and compiler engineers share vocabulary (Chen and YiRage Contributors, 2026; Chen, 2020).

Chapter 21 compiler benchmarks supply the harness RL loops need: frozen decode cells, pinned model weights, and agreed context buckets (Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Without that harness, RL trials become incomparable noise—one engineer profiles with speculative decoding disabled, another leaves it on (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026). Wire benchmark IDs into trial logs so CI can replay the exact cell that produced each fingerprint (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Sample-efficiency tricks from production: warm-start policies from grid-search winners; reuse trials across adjacent context buckets when IR shape is unchanged; early-terminate trials that violate legality or blow memory budgets before full profile (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). RL autotune is as much about *discarding bad trials quickly* as about finding good ones (Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Budget caps per layer prevent a single attention block from consuming an entire weekly GPU pool (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*,

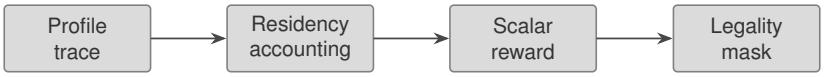


Figure 1.2: Hardware-aware reward: profile trace → buffer residency → scalar score gated by target legality.

2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

When RL policies overfit to a single SKU stepping, transfer learning across related chips (same architecture generation, different memory width) can seed exploration—but always re-profile bytes/token on the destination SKU (SemiAnalysis, 2024; AMD, 2024b; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024). Never promote transferred schedules without decode-cell A/B (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). RL autotune without transfer discipline recreates the "works on my H100" culture Part I warned against (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

1.2 HW reward

The production surprise at **hardware-aware reward design** is not peak FLOPs; it is optimizing the wrong counter (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). A reward of "minimize kernel latency" on Hopper selects schedules that win Nsight Compute yet lose decode when activations round-trip through HBM every token (Chen, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024). A reward of "maximize occupancy" encourages register-heavy tiles that shrink fusion windows and raise launches/token (Vellaisamy *et al.*, 2026; NVIDIA, 2022). Hardware-aware rewards start from tier accounting: what bytes moved between register file, shared memory, L2, and HBM/DDR for one decode step (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen, 2020).

Composite rewards work best as constrained scalars, not single numbers. A

practical template:

$$R = -\alpha \cdot \text{ms/token} - \beta \cdot \text{bytes/token} - \gamma \cdot \text{launches/token} - \lambda \cdot \mathbb{1}[\text{illegal}] \quad (1.1)$$

with SKU-specific weights: on bandwidth-saturated cards raise β; on launch-storm eager stacks raise γ (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph-capture-eligible regions add a legality bit—schedules that allocate inside captured DAGs receive $-\infty$ reward regardless of microbench speed (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Trace residency before trusting R. Reward producers (attention, MLP, norm) must feed consumers without a DDR round trip (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Silent spill reads as "mysterious" p99 regression when mean ms/token looks flat (Davies *et al.*, 2025; SemiAnalysis, 2024). Diff lowered IR or CUDA for new global allocations; count DMA edges on NPU trials like launches on GPU (AMD, 2024a,b; Chen, 2020).

Nsight Compute and roofline charts remain the auditor's tools—not the reward itself (SemiAnalysis, 2024; Davies *et al.*, 2025; Chen, 2026). Translate profiler counters into bytes moved per decode step before they enter R; otherwise teams overfit to SM occupancy (NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Chen, 2026). Chapter 1's bytes/token counter is the bridge between profiler vocabulary and autotune objectives (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Graph-mode decode adds reward terms for capture legality: schedules that introduce allocations inside captured regions should fail regardless of ms/token (NVIDIA, 2024; Chen, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). MegaKernel trials add fusion-depth terms—reward schedules that reunify MLP and residual even when graph replay already fixed launches (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016). Mode-aware rewards prevent autotune from fighting Chapter 12 execution-mode decisions (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023).

Review gate. Attach roofline or buffer accounting for the target SKU before merging any autotune winner (Chen, 2026; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Rewards must be reproducible from checked-in profile scripts—not from a engineer's laptop Nsight session (Davies *et al.*, 2025; Chen, 2020).

CPU rewards emphasize LLC residency and cross-socket traffic; mis-pinning a decode thread can dominate ms/token even when GEMM kernels are "optimal" (Goodfellow *et al.*, 2016; Chen, 2020). NPU rewards emphasize compile-time static legality and DMA depth; a schedule with perfect FLOPs but one extra DDR hop fails fleet power caps (AMD, 2024b,a). Treat *same reward formula* across targets as an anti-pattern—share methodology, not weights (Chen and YiRage Contributors, 2026; AMD, 2024a; Goodfellow *et al.*, 2016).

Example 1.2.1. Two trials tie on ms/token at BS=1; trial A lowers bytes/token by 12% with one fewer launch—hardware-aware ranking picks A even if trial B wins a GEMM roofline chart (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Penalty terms matter as much as primary metrics. Add soft penalties for register spill events, uncoalesced global loads on KV tensors, and host-side synchronizations inserted by overly conservative codegen (SemiAnalysis, 2024; NVIDIA, 2022; Davies *et al.*, 2025). On CPU, penalize cross-NUMA migrations; on NPU, penalize DDR round trips that static analysis could have folded (Goodfellow *et al.*, 2016; AMD, 2024b,a; Chen, 2020). These penalties keep RL from "winning" on ms/token while hiding residency debt that explodes at ctx=8192 (Chen, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Reward hacking appears when engineers reuse the same profile script but change warmup: steady-state ms/token improves while cold-start SLO regresses (Chen, 2026; NVIDIA, 2024; Davies *et al.*, 2025). Lock warmup protocol in CI—fixed bucket sequence, primed graph caches, identical sampling flags—before comparing rewards (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022). Fregly-style goodput analysis always states which phase is measured (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026).

Multi-objective Pareto fronts beat single scalars when leadership must choose between 5% bytes/token and 2% launches/token (Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024). Publish Pareto sets per SKU; let product owners pick operating points—compiler teams should not hide trade-offs inside opaque R (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD,

2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hardware-aware reward design is governance, not just math (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability closes the loop: every trial should emit structured logs—knob vector, legality outcome, the three decode counters, profiler deep link, and git SHA of compiler stack (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Dashboards that only show "best ms/token so far" invite reward hacking; dashboards that plot bytes/token and launches/token alongside expose orchestration regressions early (Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). On-call engineers promoting autotune winners need the same artifacts SREs use for binary rollbacks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Stability testing belongs in the reward pipeline: run promoted schedules across context buckets $\{512, 2048, 8192\}$, cold and warm starts, and at least two driver versions (Chen, 2026; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Liu *et al.*, 2024). A schedule that wins at ctx=2048 but fragments at ctx=8192 should fail promotion even if mean reward improved (Chen, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). Hardware-aware rewards without stability gates recreate the prefill/decode split Chapter 1 documented (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

1.3 YiRage auto

YiRage auto-optimization is where search meets the runtime stack from Chapters 2 and 29: ‘superoptimize‘ over multi-backend IR, ‘HardwareRegistry‘ for SKU detection, and fingerprint caches that reuse proven schedules without rerunning full RL episodes (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023). The failure mode is bolting an external autotuner onto PyTorch exports—two sources of truth for fusion legality, diverging cache keys, and CI that cannot reproduce which schedule shipped (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

YiRage binds decode iron rules into IR metadata before search starts: residency predicates, same-backend constraints, and fallback paths when a trial illegalizes on the detected chip (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; AMD, 2024b). Search proposes knob vectors; lowering validates legality; profiling writes rewards back to the fingerprint store keyed by IR hash, backend, and context bucket (Chen, 2020, 2026). That closed loop is how AI-assisted compilation stays *auditable*—every winning schedule traces to a profile artifact, not a black-box policy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Cross-hardware practice: maintain one decode cell suite per fleet tier (Hopper H100, MI300, XDNA NPU, CPU reference) and require non-regression on bytes/token before promoting a fingerprint (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen, 2020). GPU-first winners that spill on NPU should remain quarantined in backend-specific namespaces—shared YAML knobs are convenient and dangerous (AMD, 2024a; Chen and YiRage Contributors, 2026; AMD, 2024b). When ‘superoptimize‘ exhausts its budget, fall back to human-prior schedules from Part VI compiler chapters rather than shipping under-measured RL noise (Chen, 2020, 2026; Vellaisamy *et al.*, 2026).

Integration with serving stacks: vLLM/SGLang own batching and KV paging; YiRage auto targets intra-layer fusion and PersistentKernel handoff—reward functions must not double-count framework-level wins (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). DeepSeek-class MoE adds routing entropy; auto-optimization should segment experts into separate search cells or rewards absorb load imbalance (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). FlashAttention-class kernels shrink attention bytes; auto must still score MLP/residual bytes/token or reports lie (Dao *et al.*, 2022, 2023; Chen, 2026).

‘HardwareRegistry‘ and same-backend rules from YiRage docs prevent silent backend mismatch: a fingerprint compiled for CUDA must not replay on a CPU fallback without re-search (Chen and YiRage Contributors, 2026; Chen, 2020;

AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Search budgets (‘superoptimize‘ iteration caps) should scale with layer criticality—attention–MLP seams first, sampling tails last (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). PersistentKernel runtime from Chapter 29 consumes fingerprints; auto-optimization is wasted if runtime cannot load them idempotently (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. CUDA backend search explores Triton/CUTLASS tiles; CPU backend search targets oneDNN primitives; MLIR/NKI paths enforce static XDNA schedules—‘HardwareRegistry‘ selects grammar, not just clock speed (Chen and YiRage Contributors, 2026; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Example 1.3.1. Fingerprint hit: IR hash + H100 + ctx2048 reuses a prior ‘superoptimize‘ result; miss triggers bounded RL search with decode reward—cold start latency includes search unless warm caches are primed in CI (Chen and YiRage Contributors, 2026; Chen, 2026; Kwon *et al.*, 2023).

Production checklist: (1) legality oracle before profile; (2) composite decode reward with SKU weights; (3) fingerprint store with IR/backend/bucket keys; (4) A/B against human prior on bytes/token and launches/token; (5) quarantine per-backend winners (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a). That is auto-optimization as engineering discipline—not “turn on AI and hope” (Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024).

CI integration: store ‘superoptimize‘ fingerprints as build artifacts keyed by git SHA and compiler version (Chen and YiRage Contributors, 2026; Chen, 2020). When LLVM or Triton bumps change legality, invalidate caches selectively rather than rerunning full RL from scratch (Chen, 2020; SemiAnalysis, 2024; AMD, 2024b). Nightly jobs replay decode cells on representative SKUs; drift alerts fire when bytes/token moves more than agreed epsilon (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; Goodfellow *et al.*, 2016; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Human-in-the-loop remains mandatory for safety-critical deployments: RL proposes, engineers approve promotion, on-call runbooks list rollback fingerprints (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026). LLM-assisted *synthesis* of schedule hints—suggesting fusion candidates from IR text—can seed RL policies but must never bypass legality oracles (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen, 2026; Davies *et al.*, 2025; Chen, 2020). AI assists search; decode metrics retain veto power (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Chapter 24 heterogeneous deployment assumes auto winners are tagged per backend; Chapter 26 packaging assumes fingerprint stores are reproducible in containers (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). YiRage auto closes the loop between Part VI compilers and Part VIII runtime co-design (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Operator playbook summary: detect chip via registry; load fingerprint or start bounded search; validate legality; profile decode cell; compare against human prior; canary; promote or rollback (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). Each step has an owner—compiler, runtime, SRE—so auto-optimization does not become a headless cron job (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). Chapter 27 trends toward more autonomous search; this chapter insists autonomy remains subordinate to measured decode goodput (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

When external LLM tools suggest fusion patterns, treat output as *hints* fed into

the same legality and reward pipeline—never as merged schedules without profile (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). AI-assisted compilation accelerates hypothesis generation; hardware-aware rewards and YiRage integration retain veto (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). That division of labor is how industrial teams adopt ML without surrendering residency discipline (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

1.4 Key Takeaways

- **RL autotune.** Use policies over compiler knobs when search spaces explode; train on decode cells (BS=1, bucketed ctx), not prefill FLOPs alone (Vellaisamy *et al.*, 2026; Chen, 2026, 2020).
- **Hardware-aware reward.** Composite R over ms/token, bytes/token, launches/token, and legality—SKU-specific weights, shared methodology (Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b).
- **YiRage auto.** Closed-loop ‘superoptimize‘ + ‘HardwareRegistry‘ + fingerprint caches; quarantine cross-backend winners (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024).
- **Measure useful decode work.** Prefer kernels/token and bytes/token over peak FLOPs when ranking autotune trials (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026).
- **Mechanical sympathy bounds the search.** Run legality oracles before spending GPU-hours so the autotuner never explores schedules that cannot fit the target’s residency budget (Dao *et al.*, 2022; Chen, 2020). Folding residency accounting into every reward keeps the search anchored to on-chip behavior rather than to a microbenchmark the policy can game (AMD, 2024a; Chen, 2026).

1.5 Conclusion

Auto-optimization and AI-assisted compilation extend Part VI backends with search—but only decode metrics from Chapter 1 confer legitimacy (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). RL explores; hardware-aware rewards judge; YiRage integrates both without forking the toolchain (Chen and YiRage Contributors, 2026; Chen, 2020). Chapter 26 assumes these baselines when packaging production deployments; Chapter 28–30 assume fused schedules won autotune gates before framework/runtime co-design (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024a). Gate merges on buffer accounting and reproducible profile artifacts—never on a single GEMM roofline screenshot (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 2, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 3, 5, 6, 7, 8, 9, 11, 12, 13

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14